quadbit: Hand-Written FP4 Tensor-Core Kernels and a Deployment Stack for Consumer Blackwell (SM120)

Draft v0.1. Working paper. Every quantitative claim is measured on a Modal cloud RTX PRO 6000 (SM120, no tcgen05) unless tagged otherwise, and traces to a harness in `harness/` or a kernel in `cuda/`. Numbers here are copied from `docs/paper_notes.md` and `docs/standing.md`; if they disagree, those two files are the source of truth and this draft is stale.

Abstract

Consumer and pro Blackwell cards (SM120: RTX PRO 6000, RTX 5090) ship FP4 tensor cores, but the software that reaches them is thin. cuBLAS exposes dense FP4 only. CUTLASS ships a dense (example 79b) and a sparse (example 80b) NVFP4 GEMM for SM120, but the block-scaled path has documented correctness and autotuner problems in practice, and no library wraps a sparse FP4 kernel in a real deployment stack. SM120 also lacks the `tcgen05`/UMMA tensor-memory path that the datacenter B200 (SM100) uses, so every FP4 GEMM here must run through warp-level `mma.sync` and `mma.sp`.

We hand-write, in raw PTX compiled with `nvcc -arch=sm_120a`, a dense FP4 GEMM that reaches the SM120 compute ceiling and a 2:4-sparse FP4 GEMM that reaches the SM120 bandwidth roofline, having first derived the mma, `ldmatrix`, scale, and metadata bit-layouts empirically by probe-and-verify (validated to relative error 0). We then build the deployment stack around them: a weight packer, a fused NVFP4 activation quantizer, an `nn.Linear` drop-in, fused transformer-block glue kernels, and a pair-granular one-shot-plus-QAT recovery pipeline.

Since we began, the SM120 dense-FP4 baseline moved: FlashInfer's `mm_fp4` now ships a CUDA-13 `b12x` NVFP4 path plus a `cutlass` path, and on a full backend leaderboard (same card, same shapes, same fp32-reference correctness gate) they beat our hand-written two-level dense kernel by 1.35 to 2.2x. We report that honestly: quadbit does not win the dense FP4 race on SM120. The result that stands is a Pareto point no shipping library provides. First, quadbit is the only *deployed* 2:4-sparse FP4 GEMM on SM120: FlashInfer, SGLang, and vLLM ship no sparse FP4 path, and CUTLASS's sparse 80b is an unwrapped example with documented block-scaled problems. Second, our sparse kernel beats CUTLASS 80b on every shape (1.01 to 1.12x) and, more importantly, beats the *best available dense* FP4 kernel (FlashInfer `b12x`/`cutlass`) in wall-clock on every Llama-3-8B prefill shape (1.07 to 1.38x): if a weight can be 2:4-pruned, the fastest way to run its FP4 GEMM on SM120 is quadbit sparse. Third, on accuracy the deployed dense kernel is W4A4 (weights and activations both 4-bit, because the tensor core multiplies fp4 by fp4), and with a per-16 two-level NVFP4 recipe using no calibration data it costs +0.63 PPL on Llama-3.1-8B-Instruct, at or below the modelopt-calibrated reference; sparse recovery deploys at its trained accuracy through a two-level sparse kernel but stays about 1.56 PPL behind dense, so sparse is a speed-only Pareto point, not an accuracy win. Two SM120 stack facts fell out of the leaderboard and matter for anyone deploying: FlashInfer's `cudnn` FP4 backend fails on every shape (the shipped cuDNN is below the 9.14 that SM120 FP4 needs), and its fast `b12x` path collapses about 2.2x at large serving batch (tokens >= 65536), where only its `cutlass` path holds.

---

1. Introduction

FP4 is the smallest numeric format with tensor-core support on Blackwell, and on paper it promises roughly 4x the throughput of bf16 and 4x smaller weights. On the datacenter B200 that promise is largely delivered by NVIDIA's own libraries through the `tcgen05` UMMA path. On the consumer and pro cards that most people can actually buy, SM120, the situation is different: the tensor cores are present, the UMMA path is absent, and the library coverage is partial and in places buggy. That gap is the subject of this paper.

We set out to answer a concrete question: on SM120, can a hand-written kernel plus a real deployment stack turn FP4 into a usable speedup on real language models, and what does 4-bit actually cost in accuracy once you

account for the fact that the hardware quantizes activations too? Answering it required deriving the low-level layouts ourselves (the SM120 FP4 operand, scale, and 2:4 metadata layouts are not documented at the bit level), building both a dense and a sparse kernel to their respective hardware ceilings, and then quantifying accuracy on real checkpoints rather than asserting it.

Our contributions:

1. Empirically derived SM120 FP4 layouts. The dense block-scaled `mma.sync` and sparse `mma.sp::ordered_metadata` operand, scale, and metadata bit-layouts, recovered by probe-and-verify and validated to relative error 0 (Section 3).
2. A dense FP4 GEMM at the compute ceiling and the false-roofline lesson that a probe whose own load pattern is the limit will lie about the hardware ceiling (Section 4).
3. A 2:4-sparse FP4 GEMM at the bandwidth roofline, via a wide-TMA-plus-swizzle design that lifted sparse throughput +36% over a false intermediate ceiling, and that beats CUTLASS 80b on the shapes that ship (Section 5).
4. A deployment stack: packer, fused activation quantizer, `nn.Linear` drop-in, and fused transformer-block glue kernels that convert the raw GEMM wins into end-to-end block speedups of 2 to 5.8x over eager bf16 (Section 6).
5. An honest accuracy accounting: dense FP4 is W4A4 and costs +0.63 PPL with no calibration, matched to the modelopt reference; the widely quoted +0.3 is a weight-only W4A16 number that the hardware never runs (Section 7).
6. A pair-granular recovery pipeline and a clear-eyed report that on a real 8B model sparse recovery is currently data-limited and does not yet beat dense (Section 8).

---

2. Background: FP4 on SM120

The formats. NVFP4 encodes values as E2M1 (the 4-bit `{0, .5, 1, 1.5, 2, 3, 4, 6}` grid with sign) in blocks of 16, with a two-level scale: a per-16 local scale in `ue4m3` and a per-row fp32 global scale, where the global rescales the local codes into `ue4m3`'s precise range (global = rowamax / 2688, and 2688 = 448 x 6 is the product of the `ue4m3` and E2M1 maxima). MXFP4 uses blocks of 32 with a single power-of-two `ue8m0` scale; because that scale carries no mantissa, it has no per-block rounding bias, which matters for accuracy (Section 7).

What the tensor core runs. The FP4 mma multiplies fp4 by fp4. There is no weight-only FP4 GEMM in silicon. So the deployed kernel is always W4A4: both operands are 4-bit. Any accuracy number that keeps activations at 16-bit (W4A16) describes a kernel that does not exist on this hardware; it is a dequant-then-bf16-matmul path (Marlin-class), not an FP4 GEMM. This distinction drives the entire accuracy discussion.

No tcgen05. SM100 (B200) accumulates in tensor memory via UMMA. SM120 has none of that; the only route to the FP4 tensor cores is warp-level `mma.sync` (dense) and `mma.sp` (2:4-sparse), with accumulators living in the register file. This is the structural constraint behind every kernel decision below.

Measured hardware ceilings (RTX PRO 6000). Register-only mma peaks (dead-code-elimination defeated): sparse `mma.sp` m16n8k128 = 3626k GFLOP/s, dense `mma.sync` m16n8k64 = 1811k, a ratio of 2.002x (2:4 is a real 2x FLOP feature). L2-to-smem TMA bandwidth ceiling = 7.3 TB/s. Peak DRAM bandwidth = 1.46 TB/s. Shared memory = 99 KB per block, 100 KB per SM; L2 = 128 MB. Dense FP4 is compute-bound at roughly 84% of the mma peak; sparse FP4 is load-bound against a swizzle floor near 6.0 TB/s.

---

3. Deriving the FP4 layouts

None of the SM120 FP4 bit-layouts we needed are documented, so we recovered them by constructing inputs with known structure, running the mma, and checking which arrangement reproduced the reference to relative error 0.

- Scale layout. For the dense block-scaled mma, `scaleA[row r][kb]` maps to lane `(r & 7) * 4 + (r >> 3)`, byte `kb`; `scaleB[col c][kb]` maps to lane `c * 4`, byte `kb`.
- 2:4 metadata. For `mma.sp`, lane `L` maps to mma-row `(L & 1) * 8 + (L >> 2)`, half `(L >> 1) & 1`, with the kept-pair nibble encoded `idx0 | (idx1 << 2)`.
- NVFP4 dense scale. The `scale_vec::4X` `ue4m3` case uses the same A/B row-to-lane mapping as the `ue8m0` `2X` case, just with a 4-byte (four per-16) scale register. We confirmed this with a wide-range scale probe (2^-4 to 2^2), which is the strong test: a uniform-scale probe cannot tell whether the four scale bytes map to the correct k-sub-blocks; the wide-range one can. Result: relative error 0.

The `ldmatrix` story is a negative result worth recording. The only sub-byte `ldmatrix` ptxas accepts on `sm_120a` is the format-converting `m8n16/m16n16 .b8x16.b4x16_p64`, which expands each 4-bit value into its own byte. Our mma wants packed `e2m1x2`, so the expanded fragment is the wrong shape and would need 2x the registers to hold a full operand, which is fatal at the 255-register ceiling. We patched CubeCL to emit the instruction and confirmed it assembles and runs; it is simply the wrong tool for this kernel.

---

4. Dense FP4 to the compute ceiling

The dense kernel stages packed `e2m1x2` tiles through shared memory and issues a wide 2x8 grid of `mma.sync` instructions per warp, feeding 16 independent f32 accumulator chains back in as C so products accumulate across K. Those 16 chains are the instruction-level parallelism that hides the FP4 mma (OMMA) latency; the kernel is latency-and-ILP-bound, not occupancy-bound. Widening the warp tile from 2x2 to 2x4 to 2x8 climbed roughly 160k, 230k, 253k GFLOP/s, and 2x8 uses exactly 255 registers per thread with zero spills, saturating the register file (8 warps x 255 = 65280 of 65536 registers per SM). This is the classic "better performance at lower occupancy" regime.

The false-roofline lesson. An early mem-only probe suggested a ceiling that turned out to be an artifact of the probe's own narrow load pattern, which extracted only 54% of the L2-to-smem bandwidth. The lesson, which recurs throughout the project: never declare a memory roofline from a probe whose own load pattern is the limit.

Versus CUTLASS 79b. On square sizes, apples-to-apples (both cudaEvent-timed over 20 iterations, no torch dispatch), our dense kernel reaches 758 / 1220 / 1510 TF/s at 2048 / 4096 / 8192 versus CUTLASS 79b's 634 / 1222 / 1497: a win, a tie, and a win. But the square win does not generalize. On the rectangular Llama-3-8B GEMM shapes that actually run, we lose to 79b: attention 4096-cubed at 0.89x, ffn-up (N=14336) at 0.93x, ffn-down (K=14336) at 1.01x, all 79b-verified. CUTLASS's tile and schedule adapt to rectangular shapes better than our fixed tiling. Against 79b specifically the dense kernel is "competitive, slightly behind on shipping shapes" while delivering 3.0 to 3.7x over cuBLAS bf16, but 79b is no longer the baseline that matters: the leaderboard below shows FlashInfer beating us outright, so dense is not where we win.

We built and measured three persistent/stream-K variants to try to beat data-parallel on the rectangular shapes (stream-K, a true CUTLASS-style persistent cross-tile pipeline, and split-K via an f32 global reduction). All three regressed. The root cause is register pressure: the 128-register FP4 accumulator tile plus a software scheduler's cursor state pins the kernel near 254 registers and throttles the mma stream, while the hardware block scheduler already overlaps consecutive tiles for free. On SM120 FP4, the huge accumulator tile leaves no register headroom for a software scheduler, so data-parallel is at the practical ceiling.

The deployed W4A4 kernel is block-scaled, and its scale loads were an exposed stall. The 758/1220/1510 figures are the unit-scale kernel (compile-time-zero scales). The kernel that actually ships for accuracy is the two-level NVFP4 variant (Section 7), which must stream per-16 `ue4m3` scales from global memory each K-step; on SM120 those scales are 8-byte-pitch, so they cannot ride the tile TMA (which requires 16-byte strides) and were loaded synchronously between the tile `try_wait` and the mma, exposing ~500 cycles of latency every step. Double-buffering the scales and prefetching one step ahead with `cp.async` (while STAGES=2 and the 128-wide tile stay fixed) overlaps that load with the previous step's mma and lifts the deployed kernel 1.08 to 1.22x (square 8192 865 to 1055 TF/s, biggest where the k-step count is largest), at relative error 0 against the synchronous version. This narrows the block-scaled-vs-unit-scale gap without changing the mma; it is orthogonal to the 758/1220/1510 unit-scale ceiling above.

The full SM120 FP4 backend leaderboard (and why we lose the dense race). CUTLASS 79b is no longer the only competitor. FlashInfer's `mm_fp4` now exposes several NVFP4 GEMM backends, and on SM120 its `auto` prefers `b12x` (a CUDA-13-only SM120/121 kernel), then `cutlass`, then `cudnn` (`trtllm` and `cute-dsl` refuse SM120 outright). We benchmarked the deployed two-level quadbit kernel against every FlashInfer backend on one RTX PRO 6000, over a shape suite spanning square (2048 to 16384), Llama-3-8B prefill, decode small-M (1 to 128), and serving M = B*S, with an identical correctness gate: each backend quantizes the same bf16 operands with its own native quantizer and its output is scored against the fp32 reference (cosine > 0.97 to count; all NVFP4-on-random-Gaussian rows land at cos 0.991, and the* identical* cos/maxrel/mae across backends cross-validates that they compute the same math). Best-backend-per-shape, GEMM-only (effective TF/s = 2*M*N*K / wall):

shape	quadbit dense	FlashInfer best (backend)	FI / quadbit
square 8192	1045	1433 (cutlass)	1.37x
prefill attn 4096-cubed	838	1283 (b12x)	1.53x
prefill ffn-up (N=14336)	936	1374 (auto)	1.47x
prefill ffn-down (K=14336)	1017	1408 (b12x)	1.38x
serving ffn-down (M=65536)	639	1416 (cutlass)	2.22x

FlashInfer's `b12x`/`cutlass` NVFP4 kernels beat our hand-written two-level dense by 1.35 to 2.2x. The honest dense story is no longer "competitive with CUTLASS 79b"; the SM120 dense-FP4 baseline moved up and quadbit does not win it. Two structural findings from the same run: (1) FlashInfer's `cudnn` backend returns `No execution plans support the graph` on *every* shape, because the shipped cuDNN (9.10) is below the 9.14 that SM120 FP4 needs; (2) `b12x` collapses from ~1400 to ~640 TF/s once M >= 65536 (large serving batch), where only the `cutlass` backend holds ~1400, so on SM120 the fast dense backend is M-dependent. One toolchain fact frames the whole comparison: `b12x` requires CUDA 13, while quadbit's `sm_120a` block-scale mma (`kind::mxf4nvf4/block_scale/scale_vec::4X`) is *rejected* by ptxas 13 and only assembles under CUDA <= 12.8, so the two kernels cannot coexist in one container. Where quadbit wins is not dense; it is sparse (Section 5).

---

5. Sparse FP4 to the bandwidth roofline

Sparse FP4 is load-bound, so the kernel win is a memory-traffic win. The design stages arbitrary per-group 2:4 metadata and real per-block `ue4m3` scales coalesced through a full/empty async pipeline (no CTA-wide `__sync-threads`), on a shared-B 256x128 traffic-optimal tiling.

The wide-TMA-plus-swizzle breakthrough. A mem-only probe suggested a "2012k roofline"; it was false for the same reason as Section 4, narrow TMA boxes extracted only 54% of the 7.3 TB/s L2-to-smem ceiling. Loading WK=2 k128-slices per TMA (wider boxes) hits the ceiling, but wide smem rows cause `ldmatrix` bank conflicts. Swizzled TMA fixes that (A box 64B with `SWIZZLE_64B` and an `ldmatrix` XOR `off ^= ((off >> 7) & 3) << 4`; B box 128B with `SWIZZLE_128B` and XOR `((off >> 7) & 7) << 4`). Result: unit-scale sparse went from 2012k to 2731k (+36%), and the deployable kernel from 1486k to 2116k (+42%).

Versus CUTLASS 80b, the only other sparse FP4 kernel. No shipping library provides a sparse FP4 GEMM on SM120: FlashInfer, SGLang, and vLLM expose dense NVFP4 and MXFP4-MoE paths only, and CUTLASS's 80b is an unwrapped example. quadbit is therefore the only *deployed* 2:4-sparse FP4 GEMM on the platform. Against 80b, correctness-gated on 80b's own reference check (which passes at every size, so CUTLASS issue #3096's block-scaled bug does not affect this comparison), the deployed two-level sparse kernel wins on every shape: square 8192 at 1.09x (1973 vs 1807 TF/s), attention 4096-cubed at 1.08x, ffn-up at 1.01x, ffn-down at 1.12x, all 80b-verified.

The Pareto point: sparse beats the best available *dense* FP4. The reviewer-obvious result is cross-table. On the same card and shapes, quadbit's two-level sparse kernel beats not just its own dense and CUTLASS 80b, but the *fastest FlashInfer dense backend* in wall-clock on every Llama-3-8B prefill shape: attention 4096-cubed 0.100 vs 0.107 ms (1.07x), ffn-up 0.301 vs 0.350 ms (1.16x), ffn-down 0.265 vs 0.342 ms (1.29x), square 8192 0.557 vs 0.767 ms (1.38x). So if a weight can be 2:4-pruned, the fastest way to run its FP4 GEMM on SM120 is quadbit

sparse, faster than the best dense FP4 kernel anyone ships. That is the Pareto point no library currently provides. (Effective FLOP counts the pruned zeros; the honest comparator is wall-clock for a fixed GEMM problem, which is what these ratios are. The two sides were measured in separate containers, CUDA 12.8 for sparse and CUDA 13 for FlashInfer, forced by the mma/ptxas split noted in Section 4, on the same physical card at the same clocks.)

Ceiling honesty. Sparse's real advantage over our own dense FP4 is roughly 1.33x at the roofline (2012k vs 1510k deployable at 8192), not the 2x the mma FLOP ratio suggests. The hardware 2x is a datacenter-bandwidth feature we cannot reach on SM120. At 1.33x, the accuracy cost of sparsity has to be small to justify the recovery pipeline over dense, which is exactly the tension Section 8 confronts.

Throughput summary (M=N=K, vs cuBLAS bf16), speed-path ceiling:

size	cuBLAS bf16	CUTLASS FP4	dense FP4 (ours)	2:4-sparse FP4 (ours)
4096	372 TF/s	1222	1136 (3.06x bf16)	1512 (4.07x bf16)
8192	423 TF/s	1497	1556 (3.68x bf16)	2207 (5.22x bf16)
16384	405 TF/s	n/a	1645 (4.06x bf16)	1782 (4.39x bf16)

These are the speed-path ceiling numbers: MXFP4-fast dense (`ue8m0` per-32) and unit-scale sparse, the fastest variants, not the accuracy-deployed two-level kernels. The deployed two-level kernels that carry the accuracy recipe are slower and are the ones on the leaderboard in Section 4 (dense square-8192 1045, not 1556) and the sparse head-to-head above (sparse square-8192 1973, not 2207). Reporting both protocols matters: the two-level rescale that buys the accuracy result costs throughput, so the deployed sparse win over the best FlashInfer dense (1.07 to 1.38x) is measured on the *deployed* kernel, not this ceiling.

---

6. Deployment stack and operator fusion

The raw GEMMs are at the silicon ceiling, so every remaining end-to-end gain came from fusing the glue between GEMMs, removing the memory round-trips a transformer block otherwise pays in eager mode.

- **`QuadbitLinear`** is an `nn.Linear` drop-in: a torch packer reproduces the kernel's exact metadata, compression, and scale layout (verified maxrel 0.0039), fed by a fused 128-bit NVFP4 activation quantizer in a single CUDA pass. End-to-end it is 4.0 to 4.2x over torch bf16 at 8192, for any token count.
- Fused SwiGLU FFN. Quantize the activation once (shared across gate and up), concatenate gate and up into one GEMM, and fuse the epilogue (read gate and up, compute silu(gate)*up, emit the FP4-packed down-proj input and scales in one transposing pass). Cumulative versus torch bf16: unfused 2.05x, plus fused epilogue 4.45x, plus concat 4.66x at batch 2048 (2.27x over unfused), and 1.74x to 2.95x at batch 512. Numerically identical to the unfused path; a pure memory-traffic win.
- Fused RMSNorm-plus-quant (`rmsnorm_quant_k`): one read of x replaces eager rmsnorm's read-reduce-write plus a separate quant pass, and it is more accurate (no bf16 round-trip before quant). 3.7 to 4.3x over eager rmsnorm-plus-quant.
- Fused residual-add-plus-RMSNorm-plus-quant (`add_rmsnorm_quant_k`): the full block transition in one kernel. 5.3 to 5.8x over the eager sequence.

Every inter-GEMM memory round-trip in a transformer block is now a single fused pass, at zero accuracy cost (the FP4 and prune floors are preserved).

On real models. Every projection of the current frontier models tiles onto the kernel (all hidden sizes are multiples of 256 and head_dim is 128): we ran the real linear shapes of Qwen3.5-397B, GLM-5.2, MiniMax-M3, and DeepSeek-V3/R1 through the kernel, all all-tile-ok. A full fused dense FP4 decoder block on real Qwen3-8B weights, with no training, runs 2.16 to 2.19x over bf16 at block reconstruction rel 0.13.

---

7. Accuracy: dense FP4 is W4A4

Because the tensor core multiplies fp4 by fp4, the accuracy question is W4A4, not W4A16. With a per-16 two-level NVFP4 recipe using amax and no calibration data, measured through `harness/recovery_worth.py` on WikiText-2:

- Llama-3.1-8B-Instruct: 7.27 to 7.90, +0.63 PPL.
- Meta-Llama-3-8B base: 6.20 to 6.91, +0.71 PPL.
- Reference: vLLM native NVFP4 (modelopt-calibrated) is 7.97, +0.71 on the same windows.

So our own amax recipe, with no calibration, lands at or below the calibrated reference. The earlier "+2 PPL" figure was a crude per-32 single-level recipe, not W4A4's real cost; the gap was block granularity and two-level activation scaling, not calibration. The often-quoted +0.3 PPL is a weight-only W4A16 number that the hardware never runs, and we do not headline it.

At the block-reconstruction level, two-level NVFP4 reaches rel 0.097 on real Qwen3-8B with no training, versus 0.13 for the simpler and faster MXFP4 (`ue8m0`) path. NVFP4 is the accuracy path; MXFP4 is the speed path (2.15x over bf16). The single-level NVFP4 block regressed to 0.38 because the `ue4m3` scale carries a mantissa and its per-block rounding bias accumulates across the three-matmul chain; the two-level recipe (per-row fp32 global rescaling the per-16 `ue4m3` locals) is what fixes it, since MXFP4's power-of-two scale has no such bias.

> In progress (reserved slot, no number claimed). A training-free, memory-free mixed-precision refinement that keeps the most activation-sensitive layers at W4A16 and the rest at W4A4 is under evaluation. Layers are ranked on decontaminated C4 and scored on held-out WikiText-2 to avoid selection-on-test overfit, and a minimal-K sweep deploys the smallest count of bf16-activation layers that crosses the target, since each such layer is prefill compute paid for. No improved PPL is reported here; the figure will land once the ablation converges and the docs re-sync. Dense W4A4 at +0.63 PPL, zero calibration, is the accuracy result this paper stands on.

---

8. Sparse recovery and the two-level deploy fix

Blackwell FP4 2:4 is pair-granular: the `mma.sp` metadata selects at fp4-pair granularity (2 of every 4 pairs kept, not 2 of every 4 elements). This is NVIDIA's documented hardware spec, not a discovery of ours. The consequence we do contribute as a data point: on an existing element-2:4 checkpoint (`neuralmagic/Sparse-Llama-3.1-8B-2of4`), pair-granular selection keeps only about 87% of the nonzero energy, so naive reuse gives 93.6 PPL versus 7.9 for dense FP4. Element-2:4 tooling and pair-granular hardware are not interchangeable.

Our recovery pipeline retargets SparseGPT to pair-granular masks (keep the two pairs of four by `w^2 / [H^-1]^2` with Hessian error compensation), then distills from the dense teacher with the mask frozen, then runs QAT with straight-through fake-quant of both weights (exact kernel dequant) and activations. On TinyLlama-1.1B this recovers to 9.60 PPL through the real sparse FP4 kernel (dense teacher 7.53), and closing the STE-versus-kernel gap (matching the QAT activation fake-quant to the deployed quantizer bit-for-bit) was worth about 0.43 PPL.

The deploy gap and its fix. Recovery trains against a two-level fake-quant: a per-row and per-column fp32 global rescaling the per-16 `ue4m3` local scale, the same move that took dense NVFP4 from 0.38 to 0.097 block-reconstruction error. The originally deployed sparse kernel applied only the local scale, so its outputs diverged from what QAT trained against. We measured that deploy gap directly, running three matmuls of one recovered Meta-Llama-3-8B checkpoint through each path: the single-level kernel deploys at 11.89 PPL, the two-level kernel at 8.95, against a fake-quant target of 8.96. The single-level path flips 22% of top-1 next-token predictions relative to the target; the two-level path reproduces it (mean NLL delta 0.002, 92% top-1 agreement). We built the missing piece, a per-row and per-column fp32 global rescale in the sparse mma epilogue ported from the working dense two-level path. Through the two-level sparse kernel the recovered checkpoint tracks its fake-quant within 0.02 PPL: the deploy gap is closed. The rescale costs 2 to 10% of throughput (worst on wide-N shapes) and the two-level sparse kernel still beats CUTLASS 80b on every shape (1.01 to 1.13x, all correctness-verified).

The accuracy number, honestly deployable. The deployed sparse mma constrains activations to per-32 granularity

on the B operand, so the honest recipe matches the QAT activation fake-quant to per-32 rather than the per-16 the fake-quant ceiling used. With that match, Meta-Llama-3-8B recovers to 8.47 PPL through the two-level kernel, equal to its fake-quant (deploy gap closed end to end). Dense W4A4 zero-training on the same model is 6.91, so deployable sparse loses to dense by about 1.56 PPL. Extending phase-2 QAT from 2k to 5k steps did not help (it regressed to 8.96, mild overtraining against the WikiText-2 test), so 2k is the recovery sweet spot on this corpus.

The data lever did not pay. To test whether the residual gap was data-limited, we ran a full-scale diverse-corpus recovery (decontaminated C4, phase-1 to about 196M tokens). Phase-1 flattened at 10.82 PPL on the WikiText-2 test, far above the in-distribution WikiText-103 phase-1 (8.57), because C4 is out of distribution for that narrow test. Diverse-corpus scale did not improve the WikiText-2 number, so the recovery is in-distribution-bound on this metric, not simply data-starved. This is a negative result for the diverse-data hypothesis.

The honest position: dense FP4 (+0.63, zero-training) is the accuracy result. Sparse is a speed play (about 1.33x over our own dense) that now deploys at its trained accuracy, since the two-level kernel closes the deploy gap, but it still loses to dense by about 1.56 PPL on recovery and diverse data did not close that gap on the WikiText-2 metric. Sparse is an honest Pareto point for throughput-bound serving, not an accuracy win.

Can a hybrid recover most of the accuracy at some of the speed? Training-free, no. We asked whether selectively sparsifying only the least-sensitive matrices (keeping the rest dense W4A4) buys a meaningful fraction of the sparse speedup while staying close to dense accuracy. We ranked every matrix by the PPL cost of making just it 2:4-sparse (SparseGPT one-shot pair-granular prune, Hessian-compensated, no training), scored on a C4 selection set disjoint from the WikiText-2 test, then sparsified least-damaging-first and traced the held-out curve (`harness/sensitivity_sparse.py`). The result is negative: each matrix in isolation is nearly free (per-matrix delta-PPL ranges only -0.13 to +0.11), but the errors compound super-linearly when stacked. On the deployment-relevant MLP linears (dense 6.74, all-sparse one-shot 38.37), staying within +0.05 PPL allows only 3% of MLP FLOPs to go sparse (an estimated 1.008x), and even a +0.50 PPL budget reaches just 7% (1.018x); including attention makes it worse (all-sparse 162.7, attention sparsity the larger destroyer). Structurally `down_proj` tolerates sparsity best and `up_proj` worst. Because even a fully sparse model is only 1.33x over dense at the roofline, the hybrid upside is capped there regardless; a useful hybrid would require per-mask QAT recovery and would still be bounded by that 1.33x. The training-free free-lunch hybrid does not exist, and the sparse Pareto point that stands (Section 5: deployed sparse beats the best available dense FP4 on prunable weights) is not a dense-model hybrid but a whole-matrix prunability result.

---

9. End-to-end serving comparison (RTX PRO 6000)

We ran real serving engines on the same card and model family, same protocol, to place quadbit against what a practitioner would actually deploy. All rows: Llama-3.1-8B-Instruct family, CUDA graphs on (`enforce_eager=False`), distinct per-request prompts (identical prompts collapse under prefix caching and inflate prefill), prefill = B prompts of S=2048 with 1 output token, decode = B short prompts with GEN=128 (`ignore_eos`), WikiText-2 PPL on the same 16x2048 windows. Memory is the loader-reported weight footprint; both engines additionally pre-allocate a KV pool to their utilization fraction, so total device reservation is about 86 to 89 GiB regardless of quant.

Table A: real serving engines (paged attention, continuous batching, decode scheduler).

engine	quant	weights	WT-2 PPL	prefill tok/s (B=1/8/32/64)	decode tok/s (B=1/8/32/64)
vLLM 0.21	bf16	15.0 GiB	7.267	10209 / 26436 / 31559 / 46880	88 / 690 / 2599 / 4947
vLLM 0.21	NVFP4 (modelopt, cutlass)	5.66 GiB	7.974	13530 / 63056 / 78028 / 116831	131 / 1049 / 4259 / 8465

engine	quant	weights	WT-2 PPL	prefill tok/s (B=1/8/32/64)	decode tok/s (B=1/8/32/64)
SGLang 0.5	NVFP4 (auto → Flash-Infer CUT-LASS `fp4_gemm`)	~5.6 GiB	7.97 (same ckpt)	16829 / 59769 / 73414 / 109002	186 / 1491 / 5424 / 10145

Native NVFP4 W4A4 is real on SM120 through both engines: vLLM binds the `modelopt_fp4` cutlass method (not a Marlin W4A16 fallback), and SGLang's `auto` backend selects the FlashInfer CUTLASS `fp4_gemm` path (autotuned at startup, visible in its log). Against bf16, NVFP4 is 1.7x prefill and 1.7x decode at B=64 (vLLM), for 2.6x smaller weights and +0.71 PPL. SGLang and vLLM NVFP4 trade the lead: SGLang wins decode at every batch (10145 vs 8465 at B=64) and low-batch prefill, vLLM wins high-batch prefill (116831 vs 109002 at B=64).

Table B: quadbit dense FP4 prototype (full-forward, PREFILL-ONLY, no decode engine).

build	quant	quantized-linear weights	through-kernel WT-2 PPL	full-model prefill tok/s
quadbit dense two-level	W4A4 (per-16 ue4m3 + fp32 global)	3.93 GiB*	7.899	7987 (B=8, S=2048)

*3.93 GiB counts only the transformer-block linears swapped to the FP4 kernel; embeddings, lm_head, and attention stay bf16 and are not counted, so this is not a full-model on-device footprint comparable to Table A's loader figures.

Table B is deliberately not in Table A. quadbit ships a kernel plus an `nn.Linear` drop-in, not a serving stack: no paged attention, no continuous batching, no CUDA graphs, no decode scheduler, and attention runs in HuggingFace eager bf16. The prefill number is GEMM-plus-activation-quantizer bound over an eager Python forward, so it trails the serving engines' prefill by roughly an order of magnitude and must not be read as a serving-throughput result. What Table B does establish is that the deployed W4A4 path is accuracy-correct end to end: through-kernel PPL 7.90 matches vLLM's native NVFP4 7.97 to within 0.07 on the same windows, confirming quadbit's zero-calibration two-level recipe reaches the calibrated reference's accuracy.

Table C: quadbit inside vLLM under production CUDA graphs (one checkpoint). quadbit's two-level *sparse* MLP is registered as a `torch.library` custom op (`quadbit::fused_mlp`) so vLLM's V1 fullgraph compile and CUDA-graph capture include it (NVFP4 for all non-MLP linears, recovered Llama-3.1-8B-Instruct; accuracy and speed on the SAME checkpoint). The sparse path provably runs under production graphs (`SPARSE_CALLS=7264`, through-serving PPL 10.2709, not the 7.97 dense value — the accuracy check guards against a silent fall-back to dense NVFP4). This graph-vs-graph comparison is the production-representative serving result (util 0.8, WT-2 15x2048); see `docs/graph_serving_result.md`.

metric	vLLM NVFP4 (graph, production)	quadbit sparse MLP + split-K down (graph)	Δ
WT-2 PPL	7.97	10.2709	+2.30

metric	vLLM NVFP4 (graph, production)	quadbit sparse MLP + split-K down (graph)	Δ
prefill B=8/32/64 (tok/s)	66469/80825/119083	62914/77605/115069	−5.3% / −4.0% / −3.4%
decode B=8/32/64 (tok/s)	1046/4237/8384	1147/4543/8567	+9.7% / +7.2% / +2.2%

quadbit beats production dense NVFP4 on decode — the latency-critical, memory-bound serving regime — at every batch, with correct sparse accuracy. Prefill trails ~3-5% (it uses the plain down, which was never underfilled). The decode win comes from a split-K down projection; the earlier −6 to −12% decode loss was a diagnosed underfill, now fixed.

Decode diagnosis and fix (`--mode profile_decode`, CUDA events, µs/layer; kernel time is graph-invariant): activation-quant 15, gate_up sparse GEMM 52 (112 CTAs), fused SwiGLU 30, plain down sparse GEMM 109 (16 CTAs). `matmul_sp` launches `grid=(N/BN, M/2BM)`, BM=BN=128; at decode (tp=128) occupancy is set by the output dim, so gate_up (28672) fills 112 CTAs but down (4096) underfilled at 16 CTAs on ~188 SMs (~8% of the GPU). The fix is a split-K down kernel (`matmul_sp_sk`): a `gridDim.z` K-split raises the down to $16 \times 8 = 128$ CTAs, an f32 workspace accumulates the partial sums, and a `cvt_sp_2lvl_t` pass applies the two-level global scales post-reduction and transposes to `[tok, out_f]`. Down drops $109 \rightarrow 56.5$ µs (1.94×) at split=8 (cos 1.0000, relL2 $\approx$ 1e-5 vs the plain two-level down), and a serving sweep confirms split=8 beats 4 and 16 end-to-end. The op dispatches to `fused_mlp_2lvl_skdown` at decode (tp $\leq$ 128) and the plain `fused_mlp_2lvl` at prefill.

Eager-vs-eager (diagnostic ablation, not the production serving headline). The eager path's optimization story explains the kernel work: (1) a zero-copy transposed mma epilogue (`sparse_fp4_mm_2lvl_t`), bitwise- equal to `.t()`; (2) a two-level fused SwiGLU (11.13 single-level → 8.95 on base); (3) a single no-sync `fused_mlp_2lvl` removing ~64 `cudaDeviceSynchronize`/forward. These took the *eager* path to +3.7/+5.5/+5.6% prefill and +23% decode vs *eager* NVFP4 (Table D-eager / `docs/frozen_serving_result.md`). That win is launch-overhead only and does not survive once both paths are CUDA-graphed — hence Table C above, not the eager numbers, is the serving result.

Table C decode/prefill split hides the request-level answer, so we measured the crossover directly. Table C reports prefill and decode as separate throughputs; a real request pays both, so which path wins *end-to-end* depends on the decode fraction. We swept a batch x prompt-length x generation-length request matrix, both paths CUDA-graph captured, scoring total request latency per cell (TTFT from `generate(max_tokens=1)`, total from `generate(max_tokens=G, ignore_eos=True)`). Prefix caching must be OFF: with it on, vLLM V1 reuses the TTFT call's prompt KV in the total call, skips the real prefill, and hides sparse's prefill deficit — a first pass with caching on spuriously showed sparse winning all 112 cells. With it off (each request pays a real prefill), quadbit's sparse split-K FP4 MLP wins end-to-end total request latency outright in 81 of 112 regimes and ties 2 more (83/112 where it is at least as fast) vs production dense NVFP4. B=1 single-stream wins *every* regime (+3.5% to +11.6%); sparse wins at any batch once generation clears a batch/prompt-dependent boundary; NVFP4 keeps only the prefill-bound corner (high batch x long prompt x short generation, where it wins by <=3%).

Crossover boundary — min generation length for sparse to win total latency:

B	prompt=128	512	2048	8192
1	16	16	16	16
8	16	16	32	128
32	16	32	128	128
64	16	never (tie at 256)	1024	never (<=1024)

B	prompt=128	512	2048	8192

The boundary rises with batch and prompt length: as the workload becomes more prefill-bound, sparse needs a longer generation to amortize its prefill deficit. Accuracy is a constant +2.3 PPL (10.27 vs 7.97) across the whole map, so the crossover is purely a speed map. The serving claim: for interactive / low-batch and long-generation regimes, sparse FP4 wins end-to-end; the batch-prefill corner stays NVFP4's. Full matrix in `docs/crossover_result.md`.

Verification / multi-token decode does *not* favor sparse (refuted). Speculative/verification decoding processes k candidate tokens per sequence, so the MLP sees effective M = B*k rows per decode step; the natural hypothesis is that larger M favors sparse tensor-core work. We measured it and it is false: the sparse/NVFP4 decode-throughput margin* shrinks* with M (M=1: +13%, M=64: +2%, noisy and NVFP4-favorable beyond) and never expands. The split-K decode win is a small-M GPU-underfill fix that fades as M grows and NVFP4's own dense GEMM fills the machine. So sparse FP4 is best for low-M latency-sensitive decode, not throughput-oriented multi-token verification — consistent with the crossover map (sparse dominates B=1-8; the batch-heavy corner is NVFP4's).

Attacking the +2.3 PPL tax by reverse densification: no free Pareto point (Section 8 confirmed at serving scale). To close the constant tax we reverted selected MLP projections from recovered-sparse back to stock dense NVFP4, measured through the serving path. All-sparse 10.256 PPL, all-dense 7.974. Densifying `down_proj` recovers ~0 PPL (down-sparsity is accuracy-free *and* is exactly where the split-K decode win lives), while `gate_up` carries the recoverable tax (all `gate_up` dense → 9.750, -0.51, late layers cost most). But densifying `gate_up` *hurts* decode by 7 to 9%, because sparse `gate_up` was already the fast component. Reverse densification therefore trades speed for accuracy roughly 1:1 with no free knee; closing the tax needs QAT repair of the `gate_up`-dense/`down`-sparse hybrid (9.750), not placement alone. This is consistent with the training-free hybrid-placement negative result (Section 8). Full Pareto in `docs/accuracy_pareto.md`.

Phase-adaptive same-weight execution (Track 4C): built and refuted. A tempting way to erase the prefill-bound corner NVFP4 keeps is to run the *same* recovered pruned weights in two layouts by effective token count: prefill (large M) through a production dense NVFP4 GEMM (FlashInfer `cutlass`, weights materialized dense with zeros in the pruned slots), decode (small M) through the sparse split-K path. We built it. The semantics hold: dense NVFP4 over the recovered weights scores 10.30 PPL through serving, equal to all-sparse 10.27 within numerical drift, so the two layouts are interchangeable and the phase boundary is seamless. But the row loses: 39 wins and 66 losses of 105 crossover cells versus NVFP4, against all-sparse's 81 wins and 29 losses, and it flips none of the cells all-sparse already lost. The root cause is measured (`--mode phase_bench`, microseconds per MLP layer at prefill): the hand-rolled dense path (`nvfp4_quantize` plus `mm_fp4` plus SwiGLU plus `nvfp4_quantize` plus `mm_fp4`) runs about 2x native NVFP4 because its activation quant is unfused (the FlashInfer `nvfp4_quantize` of the down input alone is 517 microseconds at M=2048, more than both GEMMs combined), whereas vLLM fuses that quant into the norm and the SwiGLU through compiled passes an opaque custom op cannot reach. Decisive: the sparse fused MLP is already about 7 to 10 percent faster per layer than native NVFP4 (618 versus 661 microseconds at M=2048), so there is no faster dense MLP to swap in, and the corner all-sparse loses is attention and Amdahl bound, not MLP bound. Track 4C is a documented dead end, not a win. Full analysis in `docs/crossover_result.md` Section 4C.

The open axis: repairing the +2.3 PPL tax. Every serving result above carries the same constant accuracy tax, +2.3 PPL (10.27 sparse versus 7.97 dense NVFP4), and the crossover is purely a speed map because that tax does not move across the request matrix. Two training-free levers to close it are now measured and negative: reverse densification (this section) trades speed for accuracy about 1:1, and training-free hybrid placement (Section 8) buys almost no sparse FLOPs before errors compound. The remaining paper-upgrade axis is therefore accuracy repair that spends training. A tournament of four approaches ran on the recovered-Instruct all-sparse checkpoint: (1) zero-runtime calibration of the deployed sparse recipe, (2) low-rank residual adapters over the pruned weights, (3) activation-aware (Wanda-pair) mask repair, and (4) knowledge distillation from the dense teacher that retrains the sparse weights and the per-output down scale.

Result: distillation reduces the perplexity tax but does not recover downstream capability. Only the distillation family moved the metric. The best variant (KL-light/CE-heavy) reaches through-kernel PPL 8.86 (serving

PPL 9.10, from the original 10.27), clearing the target and keeping the entire split-K decode win intact (decode +9.7/+7.2/+2.2% at B=8/32/64, the banked figure; serving speed is weight-value independent, so the 81/112 crossover carries over unchanged). But the downstream-task check tells the real story: on ARC-Challenge, HellaSwag, PIQA, and Winogrande the repaired checkpoint is essentially unchanged from the un-repaired all-sparse model (about +0.005 accuracy on average), and the lowest-PPL variant even regressed on ARC-Challenge (0.356 to 0.348). The 2:4 sparsity removes roughly 20 points of ARC-C/HellaSwag accuracy versus dense, and WikiText knowledge distillation recovers almost none of it. The CE-heavy WikiText PPL win is largely domain overfitting. The other three families were negative: zero-runtime output calibration is neutral to harmful (per-channel affine 12.97, because an output rescale cannot fix a representational loss), low-rank adapters on frozen sparse weights stayed flat at about 10.0, and Wanda-pair masks with truncated QAT were worse than the SparseGPT baseline (13.06).

> Accuracy-repair result: PPL repaired, capability not. Distillation reduces the sparse serving tax from 10.27 to 9.10 PPL and preserves every serving win, but it does not close the downstream-capability gap that 2:4 sparsity opens (ARC-C/HellaSwag stay ~20 points below dense). Distillation on a narrow text corpus buys perplexity, not capability. The honest remaining frontier is sparse capability recovery (broad/larger-scale distillation data, or rethinking the prune target), not serving plumbing. See `docs/crossover_result.md` and `harness/repair.py`.

---

10. Distributed sparse-FP4 MoE on a large model (DeepSeek-V4-Flash)

Sections 4 to 9 establish sparse FP4 on a single dense model (Llama-3-8B) on one GPU. The obvious reviewer questions are whether the approach (i) transfers to a large model, (ii) transfers to a Mixture-of-Experts architecture, and (iii) works across multiple GPUs. We answer all three on DeepSeek-V4-Flash (284B total / 13B active, 256 routed + 1 shared expert per MoE layer, top-6, `moe_intermediate` 2048, hidden 4096, 43 layers, MLA + sparse-index attention). This is a strictly harder setting than Llama-8B: the MLP FLOPs live in hundreds of routed experts behind a router, not a single `LlamaMLP`, so the dense-MLP operator of Sections 6/9 does not attach.

The experts are MXFP4, not the config's FP8. The checkpoint's `quantization_config` advertises FP8 block-128, but that describes the dense/attention linears; the experts (`expert_dtype: fp4`) are MXFP4 – E2M1 codes packed two per int8 byte with an e8m0 (power-of-two) scale per 32-element block. We decode to bf16 (low nibble -> even element, matching the OCP/transformers convention) and verify the decode is value-exact (100% round-trip) on real weights, then prune 2:4 by magnitude and re-quantize into the two-level NVFP4 layout of Section 3. The re-quantized experts match the through-kernel path at cos 0.9999.

A segmented routed-row sparse GEMM makes MoE graph-capturable. A per-expert Python loop is correct but not CUDA-graph-capturable (data-dependent launch count) – fatal for the graph-mode serving of Section 9. We instead stack the local experts' packed weights into one tensor `[E*Mpe, ...]`, sort the routed rows by expert, pad each segment to the 128-row column tile, and pass a per-column-block expert id. A single kernel (`matmul_sp_moe`, one launch, fixed grid) reads its expert id and offsets only the weight-side indices (TMA-A coordinate, local scales, 2:4 metadata, per-row global scale) by `expert*Mpe`, keeping output-side indices local. The scheduling unit is a routed row, not an expert. Validated bit-exact against the per-expert kernel (cos 1.000000) across uniform, all-to-one, one-per-expert, and imbalanced routing at both tiny and real DeepSeek expert shapes; CUDA-graph capture and replay reproduce it at cos 1.000000. On one real layer (256 experts, real gate routing) the segmented operator matches the per-expert kernel at cos 1.000000 with zero non-finites.

Coverage. quadbit sparsifies every routed and shared expert MLP; the router, MLA attention, embeddings and lm_head stay dense NVFP4. That is ~91% of model parameters and ~80% of active linear FLOPs per token (top-6/256 experts + shared). In dense Llama-8B the MLP is ~66% of parameters, so the sparse coverage is *higher* for the large MoE, not lower – the result transfers, and transfers to more of the model.

Distributed (expert-parallel) scaling. Sharding the 256 experts across ranks (expert parallelism, the natural choice on RTX PRO 6000, which has no NVLink) and combining per-rank contributions with an all-reduce, the segmented kernel scales near-linearly: 2.17x on 2 GPUs and 4.21x on 4 GPUs of expert-kernel time, routing imbalance 1.04, with identical output checksum at every world size (correctness preserved). On PCIe the all-reduce

communication is 0.32 to 0.45 ms against 1.3 to 2.5 ms of expert compute – non-trivial but not dominant at this scale; on a no-NVLink fabric this communication share is the honest external-validity caveat, and it grows with world size.

Accuracy. The per-expert-output 2:4-FP4 error on real weights (random activations, a worst case) is cos ~0.70 – consistent with the single-model finding that 2:4 sparsity, not FP4, is the accuracy cost. Naively sparsifying every expert compounds this over 43 layers to incoherence. Section 10.1 shows this is recoverable training-free by placing the sparsity structurally (which projections, which layers, which routed slots), not by repairing weights – the per-expert repair we tried fails, but the capability-preserving operating points do not need it.

Serving integration status (an honest ecosystem boundary). In vLLM 0.24 the DeepSeek-V4-Flash NVFP4 checkpoint downloads, and its weights load across 2x RTX PRO 6000; vLLM selects the FlashInfer CUTLASS NVFP4 MoE backend and the FP8 Lightning-Indexer attention. But the model's FP8 (ue8m0 W8A8) attention/dense GEMMs have no working kernel on SM120: with DeepGEMM enabled the ue8m0 scale-factor transform asserts (`Unknown SF transformation`), and with DeepGEMM disabled the fallback CUTLASS c3x `scaled_mm` has no SM120 dispatch (`dispatch_scaled_mm`). Both paths are Hopper-targeted, so the model cannot complete even vLLM's init profiling forward on consumer Blackwell – a limitation of the FP8 attention path in today's serving stack, independent of quadbit (the MoE path that quadbit replaces is the one backend that did initialize). We therefore report the quadbit sparse MoE result where it is measured cleanly: the operator is validated standalone (segmented kernel bit-exact vs per-expert; CUDA-graph capturable; correct on a real 256-expert layer) and distributed (expert-parallel, 4.21x kernel scaling on 4 GPUs, correctness preserved), with the coverage and accuracy accounting above. The staged-`.so` injection path (compile the sparse mma under CUDA <= 12.8, ctypes-load under the CUDA-13 serve image) and the FusedMoE hook are implemented (`harness/serve_dsv4.py`). Section 10.1 removes this gate: we supply the missing SM120 paths ourselves through a vLLM plugin and serve the model end-to-end.

10.1 Overturning the ecosystem boundary, and training-free capability preservation

The "future work gated on ecosystem FP8 SM120 support" caveat above is resolved by building the missing kernels rather than waiting for them. A vLLM `general_plugins` plugin (installed in every spawned worker) supplies each absent SM120 path: block-FP8 dense/attention linears dequantised to bf16 at load; a hand-reimplemented MLA `o_proj` (inverse-interleaved RoPE + block-dequant einsum) replacing the Hopper-only `deep_gemm_fp8_o_proj`; the DeepSeek sparse-attention Lightning-Indexer logits reimplemented in bf16 (the FlashInfer sparse-MLA core *is* SM120-supported, only the indexer needed owning); and a pure-torch override of the cooperative-cluster top-k kernels that fail to launch on SM120. With these, DeepSeek-V4-Flash-NVFP4 generates coherent text end-to-end in vLLM on 2x RTX PRO 6000, and the quadbit 2:4-sparse-FP4 experts run in the live serving loop (load drops from ~84 to ~56 GiB/GPU as raw NVFP4 is freed after packing – proof the sparse path executed, not a silent fallback).

Training-free capability preservation. Downstream evaluation (400 items/task over ARC-C, HellaSwag, Winogrande, MMLU-5; dense AVG .7383) shows the accuracy cost is governed by *where* the sparsity is placed, and recovers with no weight training:

- Projection anchoring. The downstream tax lives in the gate/up projection; the down projection is nearly free. Sparsifying only down projections in the later 49% of MoE layers (c_down49) holds .7354, -0.29pt from dense on 2 GPUs, serving path and memory unchanged; the gate/up-only control at the same coverage falls to .7056 (-3.27pt). Down-only clears the -2pt bar up to 60% of layers; the cliff is at 65%.
- Route-slot. Keeping the top-2 highest-weight routed slots per token dense and 2:4-sparsifying the low-weight tail (D2) reaches ~33% active sparse expert-FLOP at -0.79pt (.7304), double c_down49's FLOP share – the dominant experts carry capability, the tail is nearly free. This needs dense and sparse weights co-resident for the same experts (dual residency), so it runs at 4 GPUs.
- A per-expert layerwise repair (local KD on each expert's surviving 2:4 weights) fails (-7pt): it trades the checkpoint's global weight consistency for local optima that compound. Structural placement succeeds where weight repair does not. Magnitude/Wanda-alone and all-expert sparsity also fail (all <= -4pt). See `docs/deepseek_final_table.md` and Figure `fig_ds_pareto` / `fig_ds_designspace`.

Transfer to GLM-5.2. To test that these are model-general structural rules and not DeepSeek idiosyncrasies, we

repeat them on GLM-5.2-NVFP4 (`glm_moe_dsa`: 78 layers / 75 MoE, 256+1 experts, top-8, Deep Sparse Attention + MLA, 432.9 GiB). GLM loads and generates coherently on 8x RTX PRO 6000 (expert-parallel); its DSA runs natively on SM120 (vLLM selects `FLASHINFER_MLA_SPARSE_SM120`), no fallback. Anchoring MoE layers 0-37 dense and sparsifying 38-74 (49.3%), held-out PPL (dense 3.171) moves by +0.209 for down-only vs +0.432 for gate/up – the same "down safe, gate/up expensive" mechanism at roughly half the tax – while route-slot D2 (top-2 dense, tail 2:4) gives the best quality and the highest sparse-FLOP together (+0.065 PPL, ~37% FLOP), mirroring DeepSeek's D2. Figure `fig_glm_transfer` places the two models side by side. Two honest limits: GLM quality here is measured by PPL only (the downstream MC harness is DeepSeek-tokenizer-specific and was not re-run on GLM), so the capability-preservation numbers of record remain DeepSeek's; and all GLM rows run eager, because the plugin's expert-parallel local-expert loop uses a host-syncing `torch.unique(...).tolist()` that is illegal under CUDA-graph stream capture – graph-capturable expert-parallel MoE is the remaining future work, not a DSA, attention, memory, or loader blocker.

---

11. Related work

FlashInfer **`mm_fp4`** is now the strongest dense FP4 baseline on SM120, and it moved while we worked. Its `auto` selects `b12x` (a CUDA-13-only SM120/121 NVFP4 kernel) then `cutlass`, and on the leaderboard (Section 4) these beat our two-level dense by 1.35 to 2.2x. It is not a free win for the ecosystem: the `cudnn` backend fails on every SM120 shape (cuDNN below 9.14), `b12x` collapses ~2.2x at large serving batch, `trtllm`/`cute-dsl` refuse SM120, and prebuilt cubins historically shipped no sm_120 targets (issue #3294), so the fast path needs CUDA-13 JIT. We benchmark against every one of its backends rather than a single number.

CUTLASS ships dense (79b) and sparse (80b) NVFP4 GEMMs for SM120 since 3.9.0. 80b is the only other sparse FP4 kernel in existence and is our sparse baseline; the SM120 block-scaled path has documented correctness and autotuner problems (issue #3096: grouped GEMM garbage output, TMA warp-specialized tactics failing to init), and, critically, no library wraps *any* sparse FP4 kernel in a deployment stack, which is the gap the sparse path fills.

Deployed inference on SM120. The Marlin W4A16 fallback (dequant FP4 to bf16 in-kernel, forfeiting the FP4 FLOPS) was reported around 50 tok/s on a 397B MoE, but for the dense Llama-3.1-8B NVFP4 checkpoint we measure both vLLM 0.21 and SGLang 0.5 binding the *native* NVFP4 W4A4 cutlass path on this card (Section 9: vLLM `modelopt_fp4`, SGLang FlashInfer CUTLASS `fp4_gemm`), not Marlin. Against those real engines our through-kernel W4A4 accuracy matches (PPL 7.90 vs 7.97); the remaining gap is serving-stack engineering, not the kernel.

Datacenter FP4 (B200, SM100, not our card). SGLang/FlashInfer/vLLM FP4 MoE reach roughly 1000 to 1260 TFLOPS using `tcgen05`/UMMA that SM120 does not have. We do not compare our SM120 numbers to these; the silicon differs.

Quantization and sparsity recovery. NVIDIA's NVFP4-QAD (arXiv 2601.20088) is the quant-recovery equivalent and recovers over 95% of FP accuracy on real Nemotron/Llama models; our sparse recovery is not yet in that league. SparseGPT, and the adjacent SQ-format and SharQ lines, are the sparse-plus-quant prior art we build on and must be distinguished from; our specific move is retargeting the mask to pair-granular 2:4 for the FP4 sparse path.

---

12. Limitations

- Serving-engine integration is built, graph-capturable, and beats production NVFP4 on decode; prefill still trails (Section 9, Table C). quadbit's two-level sparse MLP runs inside vLLM (V1: paged attention, continuous batching, decode scheduler) as a `torch.library` custom op that vLLM's fullgraph compile + CUDA-graph capture include, NVFP4 for non-MLP, on a recovered Llama-3.1-8B-Instruct checkpoint (accuracy + speed on ONE checkpoint), with correct sparse accuracy under graphs (PPL 10.2709, not the 7.97 dense value). A split-K down projection (`matmul_sp_sk`, 16→128 CTAs, down 109→56.5 μs) flips decode to a

win: +9.7/+7.2/+2.2% at B=8/32/64 vs production NVFP4. Prefill still trails ~3-5% (−5.3 to −3.4%); it uses the plain non-split-K down, which was never underfilled, and closing it needs a prefill-shape scheduling pass (stream-K / persistent tiling) that we have not yet built. The eager-vs-eager win (+5.6% prefill, +23% decode) was launch-overhead only and is a separate lever from the graph decode win. Future work: lift prefill to parity; graph-friendly SwiGLU/reduction fusion; accuracy recovery / hybrid sparsity.

- Sparse recovery loses to dense on accuracy (Section 8). The deployable sparse-recovered 8B is 8.47 PPL through the two-level kernel, about 1.56 above dense W4A4 (6.91), and diverse-corpus data (C4) did not close it on the WikiText-2 test. Sparse is speed-only on accuracy grounds.
- The deploy gap is now closed. The two-level sparse kernel (per-row and per-column fp32 global rescale in the epilogue) makes the deployed accuracy equal the trained fake-quant (8.47 == 8.47; the old single-level kernel would deploy the same checkpoint at 11.89). The fix costs 2 to 10% of throughput and still beats CUTLASS 80b.
- Dense loses the SM120 FP4 race to FlashInfer. The leaderboard (Section 4) shows FlashInfer `b12x/cutlass` beating our two-level dense by 1.35 to 2.2x. The dense kernel remains a useful zero-training W4A4 drop-in on the accuracy axis, but it is not the fastest dense FP4 on the platform. The win is sparse (the only deployed sparse FP4 GEMM, faster in wall-clock than even the best FlashInfer dense on prefill shapes), not dense.
- The Pareto win carries an accuracy tax. The sparse speed advantage only pays off where a weight tolerates 2:4 pruning, and on 8B that costs ~1.56 PPL vs dense; the leaderboard result is a speed Pareto point, not a free lunch. End to end in serving the tax is a constant +2.3 PPL (10.27 vs 7.97 dense NVFP4), and it is the same across the whole request matrix.
- The sm_120a block-scale mma is CUDA-12.8-only, forcing a staged build. quadbit's `sm_120a` block-scale mma (`kind::mxf4nvf4/block_scale/scale_vec::4X`) is rejected by ptxas 13 and assembles only under CUDA <= 12.8, while FlashInfer's `b12x` needs CUDA 13, so the two cannot coexist in one container. The deployed path compiles the `.so` in a CUDA 12.8.1 image and ctypes-loads it into a CUDA 12.9 vLLM process, staged via a shared volume (Section 9, `docs/repro_appendix.md`).
- int32 indexing caps a single GEMM. The kernels pass `M`, `N`, `Klog` and compute all shared memory and global offsets as 32-bit `int` (`cuda/sparse_fp4_lib.cu`, `cuda/dense_fp4_lib.cu`), so a single launch is bounded to problems whose addressed element counts fit in int32. Real LLM linear shapes are far below this bound, but a 64-bit indexing pass would be required before the kernels could serve arbitrarily large fused shapes.
- Accuracy repair repairs PPL, not capability. A four-way tournament (zero-runtime calibration, low-rank adapters, Wanda-pair mask repair, distillation) found only distillation moves the metric: it cuts the serving tax from 10.27 to 9.10 PPL while keeping every serving win, but the downstream-task accuracy (ARC-C/HellaSwag) stays ~20 points below dense, so the capability loss from 2:4 sparsity is not recovered (Section 9). Distillation buys perplexity, not capability; the honest open frontier is sparse capability recovery, not serving plumbing.
- No dense FP4 speed win. quadbit dense loses the SM120 dense race to FlashInfer 1.35 to 2.2x and is retained only as a zero-training W4A4 accuracy drop-in, not a speed contribution.
- MoE decode occupancy not yet measured. The segmented expert kernel is validated for correctness and scales at prefill routed-row counts (Section 10); at decode (few routed rows) its grid is small (the 16-CTA underfill regime of Section 8), and a split-K segmented variant – the mechanical analogue of the single-MLP split-K down we already ship – is future work, not yet built or measured. Because end-to-end DeepSeek-V4-Flash serving is ecosystem-blocked on SM120 (Section 10), decode latency for this model could not be exercised regardless.
- MoE accuracy recovery untried. The MoE experts are pruned 2:4 by magnitude only; calibrated SparseGPT / distillation on the experts (the dense-model levers of Section 8) are not yet applied at MoE scale. The per-expert-output tax (cos ~0.70 on random activations) is reported un-repaired.

---

13. Conclusion

On SM120, FP4 is a real speedup but the ecosystem's coverage is uneven, and it moved under us. On dense FP4 we no longer lead: FlashInfer's CUDA-13 `b12x/cutlass` kernels beat our hand-written two-level dense by 1.35 to 2.2x, and we report that plainly. What stands is a Pareto point no shipping library provides. quadbit is the only

deployed 2:4-sparse FP4 GEMM on SM120, its sparse kernel beats CUTLASS 80b (the only other sparse kernel) on every shape, and in wall-clock it beats even the best available *dense* FP4 kernel (FlashInfer `b12x/cutlass`) on every Llama-3-8B prefill shape by 1.07 to 1.38x: if a weight can be 2:4-pruned, quadbit sparse is the fastest way to run its FP4 GEMM on the platform. The accuracy axis is honest too: the deployed dense W4A4 path costs +0.63 PPL with no calibration, and sparse deploys at its trained accuracy through the two-level kernel but stays ~1.56 PPL behind dense, so the sparse advantage is speed, conditioned on prunability. The contribution is a hand-written kernel plus deployment stack that occupies a Pareto corner (sparse FP4, deployed, fastest-for-prunable) that CUTLASS, FlashInfer, SGLang, and vLLM leave empty.

Two results extend this beyond the dense single-GPU story (Section 10.1). First, on a large MoE the accuracy cost is not a fixed tax but a placement problem: sparsifying only down projections in later layers, or only the low-weight routed slots, preserves downstream capability training-free on DeepSeek-V4-Flash (-0.29pt at 49% of MoE layers sparse; a per-expert weight repair, by contrast, fails). Second, the same structural rules transfer to GLM-5.2 served on 8x RTX PRO 6000, whose Deep Sparse Attention runs natively on SM120 – down-only sparsity costs about half of gate/up sparsity there too. Both run eager; graph-capturable expert-parallel MoE and a GLM downstream sweep are the remaining work.

Appendix: reproducibility

- Kernels. `cuda/matmul_sp_wide_swz2.cu` (unit sparse 2731k), `cuda/matmul_sp_full_wide.cu` (deployable sparse 2116k), `cuda/matmul_fp4_pp_bf16.cu` (dense 1503k), `cuda/dense_fp4_lib.cu`, `cuda/sparse_fp4_lib.cu` (PyTorch-callable plus fused quantizer).
- Probes. `mma_peak`, `tma_bw`, `smemq`, `sp_*_probe`, `verify_*`, `pack_verify`, `pack_accuracy`.
- Harnesses. `bench_vs_bf16.py` (throughput), `cutlass_fp4.py` / `cutlass_sparse.py` / `cutlass_shapes.py` (CUTLASS baselines), `quadbit_linear.py` (drop-in), `recovery_worth.py` (dense W4A4 accuracy), `sparsegpt_pair.py` (one-shot), `finetune_pair.py` (recovery), `sensitivity_sparse.py` (hybrid sparse-placement sweep, training-free negative result), `vllm_nvfp4.py` (vLLM SM120 NVFP4 smoke test).
- Full chronological build log. Memory `quadbit-raw-ptx.md`.
- Exact commands, model ids, checkpoints, and the staged .so build recipe. `docs/repro_appendix.md`.
- Figure plan (what each figure shows and its data source). `docs/figure_plan.md`.
- Claims checklist (every claim to its evidence and status). `docs/claims_checklist.md`.